Project Report Template

Advanced Analytics and Applications

Team 1

Authors:
Ivan Kamal Mehieddin (7396805)
Nils Bringewatt (7443820)
Maike Katterbach (7444323)
Vitus Grofl (7380537)
Contact: mkatter1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

List of Figures

1 Introduction

1.1 Problem Description

Urban taxi demand is highly uneven across both space and time. In a city such as Chicago, trip volumes vary by neighborhood, hour of day, weekday, weather conditions, airport activity, commuting patterns, and local points of interest. For a fleet operator, this creates a practical planning problem: vehicles should be available where passengers are likely to request rides, while unnecessary idle time, empty repositioning, and inefficient charging should be reduced.

This project uses Chicago taxi trips as a proxy for urban ride-hailing demand. The core problem is to understand and predict how many trips will start in different parts of the city during a given time window. In this report, demand is therefore defined as the number of taxi pickups per spatial unit and time interval.

1.2 Business Goal

The business goal is to support better operational decisions for a taxi or ride-hailing fleet operator. More accurate knowledge of where and when demand occurs can help operators position vehicles in high-demand areas, reduce passenger waiting times, avoid unnecessary idle time, and improve the utilization of available vehicles. For an electric fleet, this planning task is connected to charging decisions: vehicles need enough energy to serve expected demand, but charging should be scheduled in a cost-efficient way.

From a managerial perspective, the project aims to turn historical trip data into practical recommendations for fleet positioning, demand planning, and charging strategy. The final value of the analysis is not only whether a model predicts demand accurately, but whether the results can inform decisions that improve service reliability and operational efficiency.

1.3 Data Mining Goal

The data mining goal is to build a spatio-temporal demand prediction task from raw taxi trip records and external contextual data. Individual trips are aggregated into panels by time window and spatial unit, and the target variable is the number of pickups in each panel cell. Calendar variables, weather data, and spatial identifiers are used as explanatory features.

The project evaluates whether machine learning methods can predict aggregate taxi demand at useful resolutions. In particular, Support Vector Machine models are trained and compared against simple time-profile baselines. Model performance is assessed using appropriate error metrics and benchmarking, with attention to whether the models perform well not only overall but also in the high-demand areas that matter most for fleet operations.

2 Data

We combine two data sources covering 2024-01-01 through 2026-05-31: Chicago Taxi Trips, the full set of reported trips pulled from the City of Chicago Data Portal (dataset `ajtu-isnz`), and hourly weather (temperature, precipitation, snowfall, wind, cloud cover) for Chicago from the Open-Meteo archive API. The taxi data cuts off at the end of May 2026 rather than the pull date because the portal has a reporting lag of several weeks — June 2026 held only 31 rows instead of the roughly 600,000 expected, which would otherwise look like a demand crash in the prediction models.

The raw taxi export contains 16,086,180 rows across 21 columns. The cleaning pipeline concatenates the 2024, 2025, and 2026 CSV exports, applies the steps below, and writes the result out as a single Parquet file. In total, 10.1% of trips (1,620,530 rows) are removed:

1. Trips are deduplicated on `trip_id` before any other filtering (775 rows removed), keeping the first occurrence. Rows without a `trip_id` are set aside first and rejoined afterward, since `drop_duplicates` would otherwise treat several genuine but ID-less trips as duplicates of each other.
2. Chicago reports location at two resolutions: `census_tract` (fine-grained) and `community_area` (77 coarse zones covering the whole city). Tracts are missing for over half of trips — not randomly, but because the city masks any tract with too few trips, for privacy. Community areas are far more complete (98.2% pickup, 91.6% dropoff), since they're only missing when a trip starts or ends outside city limits. Rather than dropping those out-of-city trips, we keep every row and add `pickup_flag`/`dropoff_flag` columns marking whether an area is present, so later spatial analyses can filter on the flags without losing the trips for everything else.
3. The centroid latitude/longitude columns silently mix resolutions: where a tract exists, they hold its centroid; where it doesn't, they fall back to the community area's centroid. We derive a clean area-to-centroid mapping (from exactly the rows where the raw coordinate already is the area centroid) and join it back as separate `*_ca_centroid_lat/lon` columns, giving a resolution-consistent coordinate for any area-level aggregation.
4. Trips missing `fare`, `trip_total`, `trip_seconds`, or `trip_miles` are dropped (34,687 rows), since no meaningful analysis is possible without them.
5. Negative `tips`/`tolls`/`extras` would be removed, though none occur in practice — zero is legitimate (e.g. cash trips report no tip). Trips below Chicago's legal minimum fare ($3.25 flag-drop) or shorter than 60 seconds are treated as test or aborted entries and removed, together with trips above the 99.9th percentile of distance (43.0 miles) or duration (165 minutes). These bounds account for the bulk of all removed rows (1,585,068 of 1,620,530).

`payment_type` and `company` are left untouched throughout; their missing values remain as `NA` and are handled per analysis.

The cleaned dataset has 14,465,650 rows and 27 columns, spanning 2024-01-01 to 2026-05-31.

3 Data Analysis Workflow

In this section, we demonstrate how to include analysis results in your report. You can either write code directly in this `.qmd` file or reference results from a separate prototyping notebook.

3.1 Prototyping in Notebooks

You can work on your analysis in the `notebooks/prototyping.ipynb` file. When you are ready to include a result, you can use Quarto's `embed` shortcode. This is a powerful feature that allows you to maintain a clean report while keeping your extensive prototyping code in separate notebooks.

To do this, you must label the specific cell in your notebook using `#| label: fig-some-name` (and optional `#| fig-cap: ...`). Then, you can embed it here like this:

Note that this requires your `.ipynb` notebooks to follow Quarto's cell-metadata syntax.

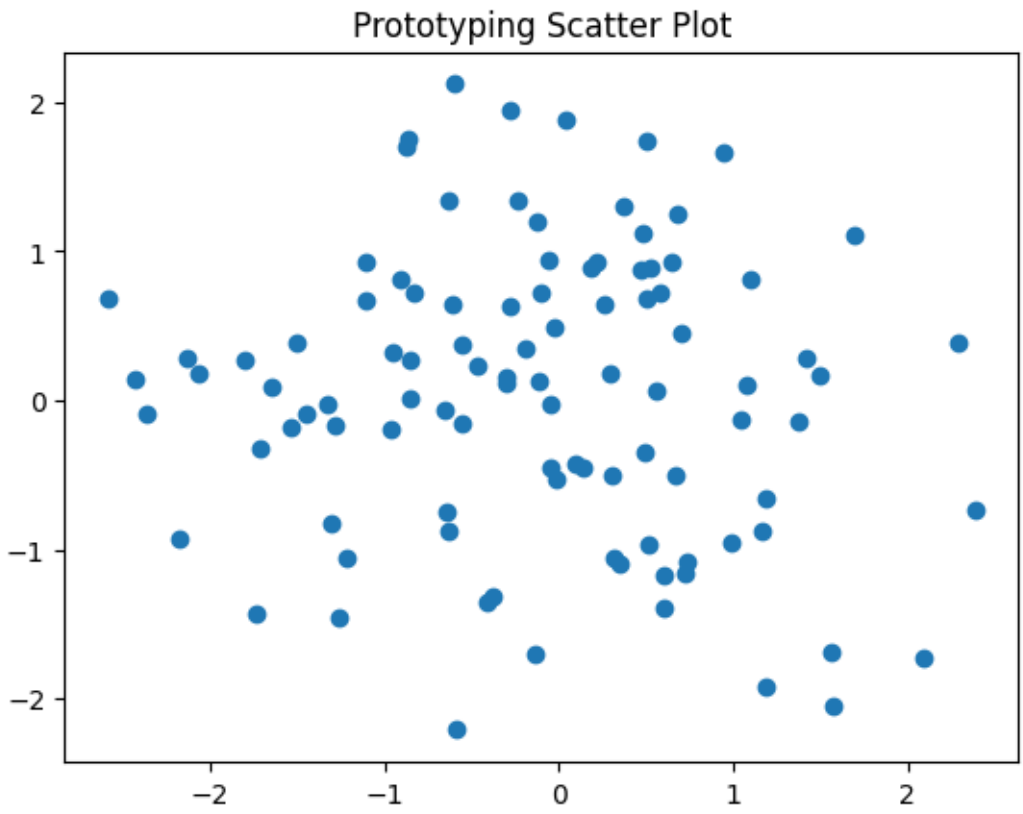

Figure 1: A plot generated in a prototyping notebook and embedded in the report.

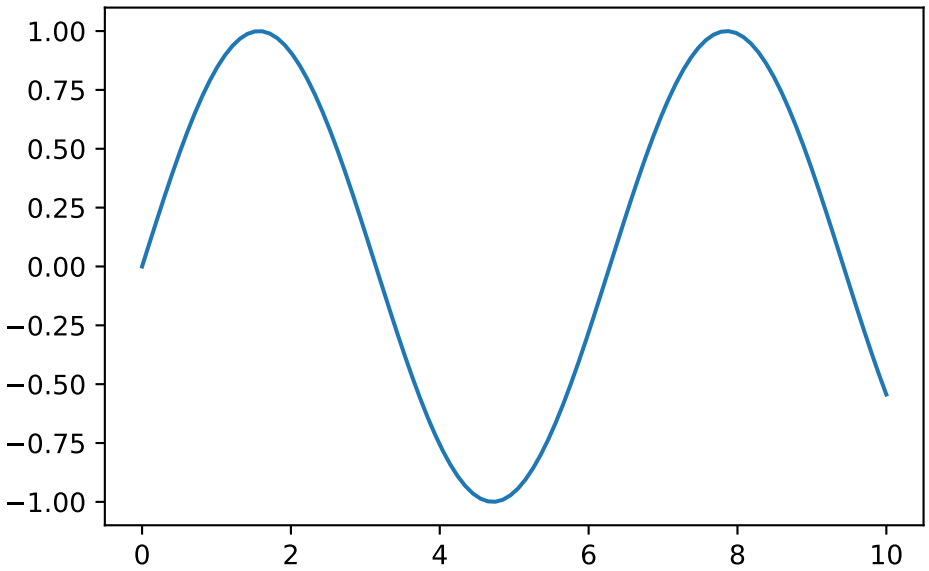

Figure 2: A sample plot generated directly in the report.

In the PDF version, the code above is hidden by default to keep the report concise and focused on the results. In the HTML version, the code can be expanded using the "Code" button.

4 Demand Prediction

We predict taxi pickup demand, defined as the number of trips originating in a spatial unit within a fixed time window. Two model families are used: Support Vector Regression (SVR) and feedforward neural networks (NN). Both are evaluated across several spatio-temporal resolutions: community area at 4-hour and 1-hour windows, and census tract at 4-hour windows. Results below are organized by resolution so the two model families can be compared directly wherever both are available; a synthesis of findings across resolutions and models follows in Section 4.4.

4.1 Modeling Approach

Both model families share the same task framing: a single **pooled model per experiment**, trained across all spatial units at once, with spatial identity encoded as one-hot fixed-effect dummies rather than fit as separate per-unit models. This scales to hundreds of census tracts without multiplying training time, and lets panel-construction and evaluation code be reused across resolutions. Two families were tried for complementary reasons: SVR (linear and kernelized) is cheap to fit and, in its linear form, interpretable via coefficients; a feedforward network can in principle learn non-linear interactions between spatial identity and time features that a linear model cannot without hand-built interaction terms, at the cost of a larger tuning surface and no direct coefficient interpretation.

4.1.1 Why Pooling Is Hard

The central obstacle every model in this section has to overcome is that demand is not one distribution, it is dozens (or hundreds) of very different ones. Per-unit demand varies roughly three orders of magnitude across community areas, and the same imbalance only gets sharper at the finer-grained census-tract level, where far more units compete for the same total demand. Each unit also has its own daily and weekly rhythm, not just its own scale. A pooled model has to represent all of that with one set of parameters.

This is a genuinely hard bind for SVR specifically. `LinearSVR` fits a single global coefficient vector, so it can only find one compromise direction that trades off across every spatial unit at once; it has no mechanism to give a high-demand downtown area its own scale and its own rush-hour shape independent of a quiet residential area's. A kernel SVR (RBF, polynomial, sigmoid) could in principle represent that kind of heterogeneity through non-linear interactions between spatial identity and time features, but its training cost scales quadratically to cubically in the number of rows, which puts the full multi-hundred-thousand-row pooled training set out of reach. In practice this forces kernel variants onto a small subsample (10,000 rows), which is nowhere near enough to represent dozens of areas' worth of individual dynamics and, as the results below show, produces tuning that does not hold up on the real test set.

The profile baseline sidesteps this problem entirely by construction: it is already a separate historical profile per spatial unit, so it never has to compromise across areas the way a pooled model does. That is the main reason it is such a persistently hard target to beat throughout this section, not any particular weakness of SVR or the NN as an algorithm.

Both use the same exogenous feature set: weather (temperature, precipitation, snowfall, wind, cloud cover), cyclical time encodings (sine/cosine pairs for hour-of-day and day-of-week,

two harmonics each to capture the bimodal morning/evening rush), month, and weekend/holiday indicators. Per the constraints of this course, no lagged or autoregressive demand features are used. Only inputs known in advance of a prediction are included. Both regress on `log1p(count)`, with predictions back-transformed via a clipped `expm1`, since raw demand is heavily right-skewed (median per-unit demand spans roughly three orders of magnitude); without this transform, squared-error loss would be dominated by a handful of high-demand windows.

At census-tract resolution, one-hot spatial dummies for all 878 tracts would be impractical, so tract-level SVR instead reuses the 76 community-area dummies as a coarse spatial layer and adds tract-level covariates as a fine layer: POI count, tract area, and distance to six empirically-derived demand hot spots (identified separately via a Gaussian Mixture Model over pickup coordinates).

4.2 Validation Strategy

Both families use a chronological holdout, training on 2024-01-01 through 2025-12-31 and testing on 2026-01-01 through 2026-05-31, over a complete, zero-filled grid of every spatial unit crossed with every time window. This means evaluation includes genuine zero-demand periods, not only observed trips. SVR hyperparameter search uses `KFold` (3, shuffled): since no lagged features are used, there is no autoregressive leakage risk a chronological split would guard against, and shuffled folds give an even seasonal mix. The NN training procedure splits the training period randomly 80/20 into a fit set and an early-stopping set for the same reason.

A **profile baseline** serves as the reference every model is judged against: historical demand per (hour, weekday, month), computed separately per spatial unit.

Two metrics are reported. The **skill score**, $1 - \mathrm{MAE}_{\mathrm{model}}/\mathrm{MAE}_{\mathrm{baseline}}$ per unit, is a ratio, so each unit's own demand scale cancels out, making it comparable across units *and* across resolutions. It is aggregated demand-weighted (headline, matching a fleet-allocation use case) and as an unweighted median (a guard rail against a model that only wins on a few large units). Skill score is currently computed for SVR only. The **weighted RMSE** is an absolute, trips-per-window number reported by both families: pooled RMSE with each row weighted by its unit's train-period mean demand. It is therefore the metric used for direct SVR-vs-NN comparison below. Plain, uniformly-weighted pooled R^2/MAE/RMSE are reported only as a sanity check: demand varies roughly 1000-fold across units, so unweighted pooled numbers are dominated by the busiest few and hide poor performance elsewhere.

4.3 Results

4.3.1 Community Area, 4-Hour Resolution

SVR. The profile baseline is a much harder reference per-area than pooled numbers suggest: pooled R^2 is 0.933, but median per-area R^2 is only 0.304, with 14 of 77 areas negative, mostly low-demand areas where near-zero counts leave little variance for any model to explain. This is the heterogeneity problem from above showing up directly in a single number: a pooled R^2 is dominated by the handful of highest-volume areas and says almost nothing about the typical area. This gap is exactly why skill score, not pooled R^2, drives the comparisons below. A pooled `LinearSVR`'s demand-weighted skill is **-0.75**: it systematically underperforms the baseline specifically in the highest-demand areas, despite winning on a majority of areas by unweighted median skill (-0.03). This is the linear model's single global coefficient vector paying for its lack of per-area flexibility exactly where it costs the most, since every area contributes an equal number of training rows regardless of demand, so the shared coefficients fit mostly to the

low-demand majority. Refitting with sample weights proportional to $\sqrt{\text{median demand} + 1}$ raises demand-weighted skill to -0.61 but pushes 67 of 77 areas negative (up from 48): reweighting toward the areas the linear model handles worst comes directly at the expense of the areas it handled comparatively well, a trade-off, not a fix, because one global coefficient vector still cannot serve both regimes at once. Kernelized SVR (RBF, polynomial, sigmoid) is the other horn of the bind: it could in principle represent that per-area heterogeneity, but tuned only on a 10,000-row subsample since exact kernel methods scale quadratically-to-cubically in rows, and proved highly unstable under cross-validation as a result: a configuration scoring cross-validated $R^2 = 0.088$ reached only $R^2 = -0.032$ on the real test set. This is the one place R^2 is the right diagnostic here, since the instability is specifically about what `GridSearchCV` optimized. Fully separate per-area models (no pooling at all) for the 20 highest-demand areas still mostly lose to baseline; the single configuration across the entire study with positive demand-weighted skill is a tuned per-area RBF SVR, at **+0.023**, and it is no coincidence that the one configuration that wins is also the one that abandons pooling altogether; 12 of those 20 areas are still individually negative.

NN. Restricted to areas with median demand ≥ 10 trips/window, no network configuration beats the baseline, and more capacity makes results worse, not better. The baseline's weighted RMSE is 150.9 (plain RMSE 76.4); a shallow network (one hidden layer) reaches plain RMSE 77.1 with cyclical time encoding and 79.9 with one-hot, both close to, but not beating, the baseline. A tuned deep network (two hidden layers, dropout, weight decay), selected from a one-factor-at-a-time search over regularization and width, performs *worse* still (RMSE 82.1): additional model capacity does not help. Permutation importance shows time-of-day features dominate for both encodings and weather contributes almost nothing. Demand here is driven by the clock, not exogenous conditions.

Shared finding. Neither family beats its baseline where demand is largest, and for the NN, added capacity makes results worse rather than better. A direct SVR-vs-NN weighted-RMSE comparison, and reconciliation of the scope/baseline differences noted in Section 4.2, is pending a fresh run of both notebooks against the reformatted metric.

4.3.2 Community Area, 1-Hour Resolution

SVR. This resolution is a deliberate scaling stress test: same 77 areas, roughly 4x the rows, heavier zero-inflation, and a harder baseline too, since it now resolves 24 hourly buckets instead of 6 (baseline weighted RMSE 39.46). A pooled `LinearSVR`'s demand-weighted skill is **-1.016** (weighted RMSE 76.96), a worse deficit than the 4h resolution's -0.75 for the same model type. Finer time resolution does not ease the heterogeneity problem, it sharpens it: the same 77 areas now need to be told apart across four times as many hourly buckets, which only widens the gap a single global coefficient vector has to bridge. Kernel comparison and tuning on a 10,000-row subsample favor RBF: after tuning, RBF reaches demand-weighted skill -0.331 (weighted RMSE 54.53), polynomial reaches skill -0.361 (weighted RMSE 56.14): the best 1h configurations, though both still negative. Demand-tiered and per-area models were attempted but dropped at this resolution: per-area `GridSearchCV`'s parallel fan-out, on top of the larger 1h panel, exceeded available memory.

4.3.3 Census Tract Resolution

Only 44.6% of trips carry a true tract-level location; the rest are masked to their community area's centroid for privacy whenever a trip's originating tract had too few trips in that period. Masking is far from uniform: retention ranges from 0% to 70.9% by community area, with a clean gap in the distribution between roughly 5% and 7%. Areas below that gap have too little

tract-precision data to support any tract-level model at all. This is not a modeling choice but a data-availability ceiling, leaving **179 of 878 tracts, across 11 community areas**, in scope; the remaining areas fall back to the community-area-resolution model. Within scope, counts are built from tract-precision trips only (never the masked fallback points, which would otherwise spike demand artificially onto whichever tract contains a community area's centroid) and scaled by each area's inverse retention rate to estimate total demand. Given the smaller per-unit training data at this resolution, only a pooled `LinearSVR` is fit; kernel SVR is not attempted, following the same compute-cost reasoning established at community-area resolution, now at an even smaller per-unit scale. No NN model has been attempted at this resolution.

SVR. The profile baseline is strong here too (pooled $R^2 = 0.860$), though the per-tract split is noisier than community-area's: median per-tract R^2 is only 0.170, with 38 of 179 tracts negative, the same pooled-versus-per-unit gap seen at community-area resolution, now wider because there are more than twice as many units to average over. Pooled `LinearSVR` fares far worse than at coarser resolutions: unweighted median skill is **-0.259**, with 90 of 179 tracts negative, pooled R^2 collapses to 0.026, and weighted RMSE more than doubles the baseline's (355 vs. 133). This is the clearest illustration in the whole study of what a single global coefficient vector does when it cannot represent per-unit heterogeneity: rather than fail gracefully, it gives up on time altogether and settles for representing which tract a row belongs to. Feature-coefficient analysis confirms this directly: GMM hotspot-distance features, which take one fixed value per tract, carry 99.6% of total |coefficient| mass despite being only 6 of the 25 continuous features, while time/calendar and weather coefficients are effectively zero. The model has learned each tract's average demand level from its distance to the nearest hot spot rather than any time-varying signal. Actual-vs-predicted plots for the top-3 tracts confirm this directly: predicted demand sits flat near zero across an entire evaluation week while actual and baseline demand both trace the expected daily cycle. With 179 pooled units instead of 77, between-unit demand variance dominates the loss even more than at community-area resolution, and the tract-level hotspot-distance covariates give the optimizer an easy per-unit shortcut, at the cost of the temporal signal the model exists to capture.

4.4 Cross-Cutting Discussion

Across every SVR variant tried at community-area resolution (pooled, demand-weighted, demand-tiered, per-area) and across both NN architectures, the profile baseline is a surprisingly strong reference specifically where demand is largest. The common thread is the heterogeneity problem set out under Modeling Approach: demand differs by orders of magnitude and by shape across spatial units, and every pooled model here, linear or kernelized, either is not flexible enough to represent that difference or cannot be trained at full scale to learn it. For SVR, this is compounded by an interaction between the `log1p/expm1` transform and a squared-error-adjacent loss, which amplifies error multiplicatively: a small log-space miss becomes a large absolute error exactly for the highest counts, a penalty the baseline never incurs since it predicts directly on the raw scale. For the NN, added architectural capacity makes results worse, not better, and permutation importance shows weather contributes almost nothing, suggesting the exogenous feature set carries little signal beyond what the calendar-based baseline already captures.

Comparing SVR across resolutions, demand-weighted skill for matching model types is worse at 1h than at 4h (-1.016 vs. -0.75 for pooled `LinearSVR`; -0.331 vs. an untested comparison point for tuned RBF), suggesting finer time resolution makes the high-demand gap harder to close, not easier. This is plausibly due to thinner per-window signal and heavier zero-inflation.

4.5 Practical Implications for Fleet Allocation

The heterogeneity problem is not just a modeling inconvenience, it also points toward a more realistic deployment strategy than the one tested here. This study tried to solve the hard version of the problem: one model, fit once, that works for every spatial unit at once, including the many low-demand areas where near-zero counts leave little for any model to learn and contribute little to a fleet-allocation decision anyway. A business does not need that. It needs good predictions specifically where trip volume is high, since that is where allocation decisions have the most impact and where the demand curve already shown to vary roughly three orders of magnitude across units matters most.

The one result in this entire study with positive demand-weighted skill, the tuned per-area RBF SVR on the 20 highest-demand community areas (**+0.023**), is also the only one that abandons pooling in favor of a dedicated model per area. That is consistent with the framing above: once an area is modeled on its own, its training set no longer has to be shared with areas of a completely different scale and shape, the linear-versus-kernel-cost bind eases because each per-area training set is far smaller than the pooled one, and kernel methods become tractable without the subsampling that made the pooled kernel results unstable. The practical recommendation this suggests is to not chase one pooled model across all 77 (or 179, or 878) units, but to identify the small set of highest-demand areas that account for most of the fleet-allocation value, fit each one its own model, and leave the long tail of low-demand areas to the simple profile baseline, which already handles them about as well as anything tested here.

4.6 Limitations & Outlook

- **SVR and NN are not yet evaluated on matched scope**: differing demand thresholds and baseline definitions (mean vs. median) mean cross-family numbers above are directional, not exact, until reconciled.
- **Skill score is SVR-only**; weighted RMSE is the only metric currently common to both families.
- **Kernel-SVR tuning is unstable** under the subsample-then-tune approach required by its quadratic-to-cubic training cost. This is a property of the tuning procedure, not of kernel SVR itself.
- **Pooled SVR at census-tract resolution underperforms its baseline** (median skill -0.259, pooled $R^2 = 0.026$ vs. baseline 0.860): GMM hotspot-distance features absorb 99.6% of coefficient mass, so the model effectively predicts a constant level per tract and ignores time entirely. This is worth revisiting with per-unit time interactions rather than treated as a data-availability issue.
- **No rolling-origin backtest** for either family: a single chronological holdout is used throughout.

4.7 Appendix: Detailed Tables and Figures

5 Intelligent Charging with Reinforcement Learning

5.1 Problem Setup

The final part of the project studies a smart-charging problem for an electric taxi. The taxi returns to its home charger at 14:00 and leaves again at 16:00, which creates a two-hour charging window. The agent makes one charging decision every 15 minutes, resulting in eight decisions per episode. At departure, the vehicle must have enough energy for the next shift. If the final state of charge is below the required energy demand, the episode receives a large shortage penalty.

The task is relevant for an electric taxi fleet because charging decisions are not only a technical battery-management problem, but also an operational cost problem. Charging too slowly can leave the vehicle undercharged, while charging too aggressively can create unnecessary cost. A useful policy therefore needs to balance reliability and cost.

5.2 Markov Decision Process

We formulate the charging task as a finite-horizon Markov Decision Process. Each episode represents one daily charging window from 14:00 to 16:00.

The state includes the current state of charge, the current time step, the required departure energy, and the battery capacity. In the price-aware extension, the current electricity price and the mean remaining price are added to the state. The action space is discrete and consists of four charging levels: no charging, low charging, medium charging, and high charging. In the implemented notebook, these are represented as 0 kW, 5 kW, 11 kW, and 22 kW.

The transition function updates the state of charge according to the selected charging power and the 15-minute interval length:

$$SOC_{t+1} = \min(C, SOC_t + p_t \cdot 0.25)$$

where C is the battery capacity and p_t is the selected charging power in kW. The reward is the negative charging cost in each step, plus a large terminal penalty if the final state of charge is below the sampled energy demand. This structure encourages the agent to charge enough to satisfy the departure requirement while avoiding unnecessary charging cost.

5.3 Simulation Environment

The environment is implemented using `gymnasium`, which provides a standard interface for reinforcement learning. At the beginning of each episode, the battery capacity, initial state of charge, and required departure energy are sampled. The energy demand follows a normal distribution with mean 30 kWh and standard deviation 5 kWh, clipped to feasible values.

Two versions of the environment are evaluated. The first version uses a simplified cost function and serves as a controlled baseline setting. The second version extends the problem with real electricity prices from EPEX day-ahead data. In this price-aware setting, charging costs depend on the actual amount of energy charged and the electricity price in the corresponding time step. This extension makes the task more realistic because it allows the agent to shift charging toward cheaper intervals when enough time remains.

5.4 Learning Algorithm and Baselines

The agent is trained with a Deep Q-Network (DQN). DQN approximates the action-value function $Q(s, a)$ with a neural network and learns from repeated interaction with the simulated environment. The notebook trains the agent over 200,000 time steps and evaluates the learned policy on 1,000 paired test episodes.

The main comparison baseline is a greedy charging heuristic. This heuristic ignores prices and computes how much energy is still missing relative to the departure requirement. It then selects the smallest charging power that can cover the remaining demand within the remaining time. This rule is simple, reliable, and operationally intuitive. For a final version of the analysis, this baseline should be complemented by a true flat-rate strategy, such as charging at a fixed power level until the target energy is reached.

For a subset of episodes, a brute-force optimum is also computed. Since there are only four actions and eight time steps, all $4^8 = 65,536$ possible charging plans can be evaluated for a given episode. This provides a useful benchmark for judging whether the learned policy is close to the best possible charging plan.

5.5 Results

In the simplified cost setting, the greedy heuristic performs very strongly. It reaches a 100% success rate and is equal to the brute-force optimum on the evaluated subset. The DQN reaches a lower success rate of 96% and a substantially worse mean reward. This indicates that, under the simplified cost structure, reinforcement learning does not add value over the simple rule-based policy. The problem is structurally easy: if prices are almost constant, the best strategy is simply to charge enough to meet the known energy requirement.

In the price-aware setting, the DQN learns to react to electricity prices. Among episodes in which both DQN and greedy succeed, the DQN achieves lower average charging costs than the price-blind greedy heuristic. The notebook reports an average cost reduction of 0.0388 EUR per successful day, or approximately 4.7%. However, this cost saving comes with a lower success rate: the DQN succeeds in 95.7% of episodes, while the greedy heuristic succeeds in 100%. Because shortage is operationally costly, the lower reliability is an important limitation. When the terminal shortage penalty is included in the reward, the greedy heuristic remains the more robust policy.

The policy visualizations suggest that the price-aware DQN tends to charge more aggressively when the current price is low or when the state of charge is below the required demand. When prices are high and sufficient time remains, the agent is more likely to wait. This behavior is consistent with the intended smart-charging logic, but the reliability gap shows that the learned policy still requires further tuning before it can be recommended for operational use.

5.6 Operational Implications

The results suggest that reinforcement learning is most useful when the charging environment contains meaningful price variation. Under flat or nearly flat tariffs, a simple rule-based strategy is sufficient and easier to explain. Under dynamic tariffs, a price-aware policy can shift charging to cheaper periods and reduce energy cost, but only if it preserves a very high success rate.

For a fleet operator, the key trade-off is therefore cost versus reliability. A policy that saves a few cents per day but occasionally leaves the taxi undercharged is not acceptable

without additional safeguards. A practical implementation should either use a stronger shortage penalty, enforce a hard minimum state-of-charge constraint, or combine the learned policy with a rule-based safety layer.

5.7 Limitations and Next Steps

The current notebook provides a solid proof of concept, but several improvements are needed for the final analysis. First, the cost function should be aligned with the assignment's quadratic charging-cost formulation, or the deviation should be clearly justified as an extension. Second, the evaluation should include a true flat-rate baseline in addition to the greedy heuristic. Third, the sensitivity analysis should explicitly vary the expected energy demand, for example by comparing scenarios with 25 kWh, 30 kWh, and 35 kWh mean demand. Finally, the price-aware environment should be made fully reproducible by loading stored price windows locally instead of relying on a live API request during execution.

5.8 Appendix: Detailed Tables and Figures

6 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

7 Conclusion

The conclusion should summarize your findings and suggest future work.

References

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.